SIMATS ENGINEERING

CSA4712 – DEEP LEARNING

CO3 – ASSESSMENT 3

RECURRENT AND RECURSIVE SEQUENCE MODELS

Student A	B Akshaya • 192424336
Student B	Name and registration number of partner: _____
Group	Pair work (2 students), 50 marks
Submission Date	29.10.2026
Scenario	Industrial sensor-sequence fault and event prediction

Assignment overview

This assignment studies how recurrent and recursive neural networks represent, evaluate, tune and debug sequence data, using industrial sensor streams (vibration, pressure and temperature readings) that must be classified for faults or used to predict a future event. Every question follows the pair-work structure of the brief: shared group work, the Student A part, the Student B part and a joint answer, each with a worked table, a code excerpt and a notebook-style output panel. The Keras excerpts are templates to run in Google Colab; the numbers and plots come from an equivalent NumPy implementation of Simple RNN, GRU, LSTM and recursive networks whose gradients were verified against finite differences (maximum difference about 10^{-10}). All data are synthetic and all panels are labelled as simulations, so the results illustrate method and behaviour and are not measurements from a real plant.

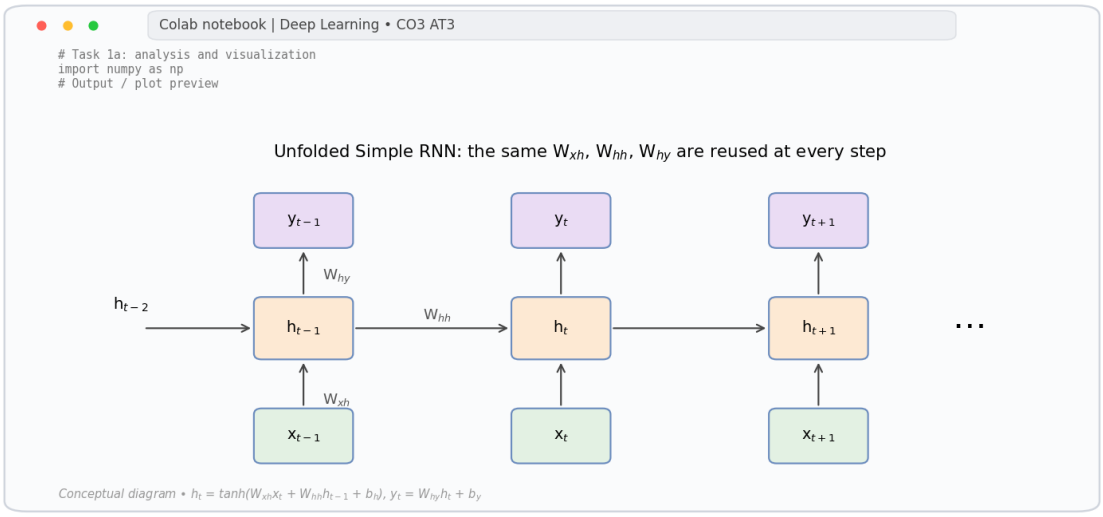
Q1 — Understanding sequence representation and recurrent processing (6 marks)

Part A — Shared group work

Dataset and task. The pair uses an industrial sensor stream. Two tasks are built from it: **forecasting** (a noisy vibration signal made of two periodic components, period 12 and period 7, predicting the next reading) and **sequence classification** (30-step, 2-sensor sequences labelled “fault” or “normal”). For forecasting, an **input sequence** is a window $x_1 \ldots x_L$ of L standardized readings, the **target** is the next reading x_{L+1}, the **sequence length** is L and the **feature dimension** is d = 1 (d = 2 for the fault data), so a batch has shape (batch, L, d). A Simple RNN with 16 units has $16 \cdot d + 16 \cdot 16 + 16 + 16 + 1 = 305$ parameters for d = 1, independent of L. The model was implemented, gradient-checked and tested; the hidden state is updated at every time step and only the final state feeds the output layer (many-to-one).

Student A — sequence design and mathematical analysis

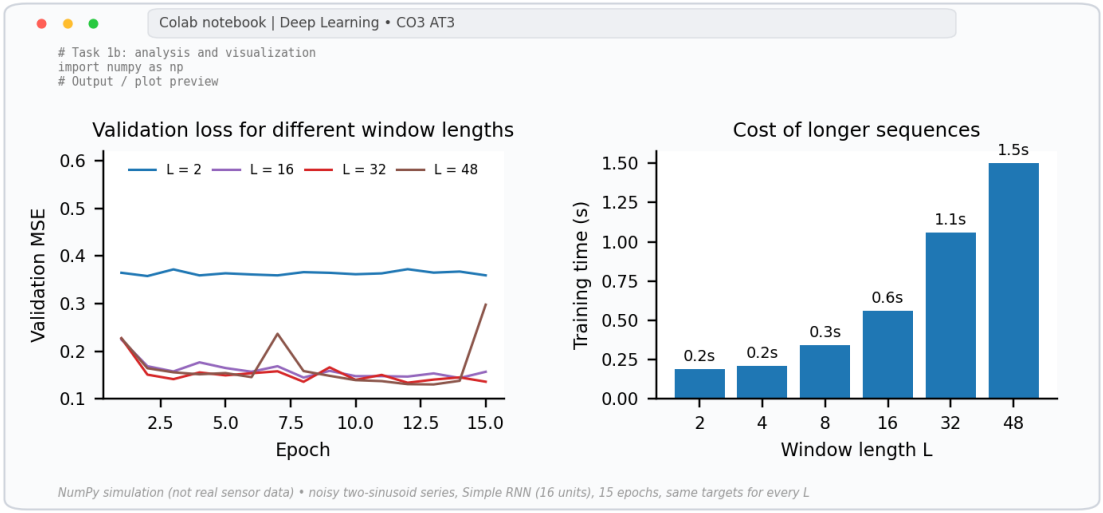
Representation. A series $s_1 \ldots s_N$ is cut by a sliding window: sample i is $(s_{i-L}, \ldots, s_{i-1})$ with target s_i; values are standardized with training statistics and windows are split in time order (70% / 15% / 15%) so that no future data leak into training. **Roles.** x_t is the input vector at step t; h_t (a 16-dimensional vector) is the hidden state, a learned summary of $x_1 \ldots x_t$; y_t is the output at step t (only y_L is used here). **State equation:** $h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$, $h_0 = 0$, and $y_t = W_{hy} h_t + b_y$. Because h_t is computed from h_{t-1}, which was computed from h_{t-2} and so on, the final state is a function of the whole window; the unfolded structure below shows the same three matrices reused at every step. **Sequence length.** A longer window gives the model more context (a period-12 signal cannot be identified from 2 points) but increases computation linearly with L and lengthens the path along which gradients must travel. **Long-term dependency.** If a pressure spike at machine start-up decides a failure 200 steps later, learning needs the gradient to pass through 200 multiplications by W_{hh} and $\tanh' \le 1$, so it vanishes (or explodes) and the dependency is difficult to learn.



Student B — experimental investigation

The window length was varied over L = 2, 4, 8, 16, 32 and 48 with the same network, seed, targets and 15 epochs, recording training loss, validation loss and wall-clock time. **Observation.** Validation MSE falls from 0.359 (L = 2) to 0.136 (L = 32), far better than the persistence baseline of 0.516, while training time grows from 0.19 s to 1.06 s. At L = 48 the validation MSE *rises* to 0.298 and directional accuracy drops to 0.739. Training loss also rises (0.364 compared with 0.143 at L = 32), so the loss of quality is not overfitting but an optimization difficulty of the Simple RNN over 48 steps within a fixed epoch budget, consistent with unstable or vanishing gradients. **Conclusion:** increasing sequence length improves performance until the window covers the signal’s periods and then degrades it, while cost keeps increasing.

Window L	Train MSE	Val MSE	Directional acc.	Time (s)
2	0.399	0.359	0.713	0.19
4	0.288	0.250	0.783	0.21
8	0.210	0.182	0.815	0.34
16	0.165	0.157	0.845	0.56
32	0.143	0.136	0.849	1.06
48	0.364	0.298	0.739	1.50

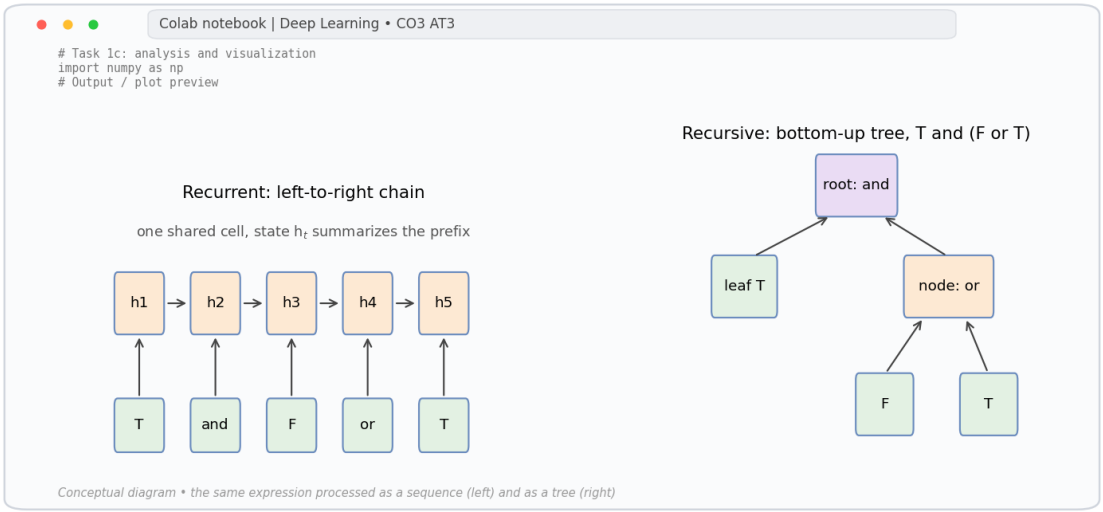



```
model = keras.Sequential([layers.Input(shape=(L, 1)), layers.SimpleRNN(16), layers.Dense(1)])
model.compile(optimizer=keras.optimizers.Adam(0.01), loss="mse")
hist = model.fit(X_tr, y_tr, validation_data=(X_va, y_va), epochs=15, batch_size=32)
print(hist.history["loss"][-1], hist.history["val_loss"][-1])   # repeat for each L
```

Part B — Recurrent vs recursive processing

Student A — recurrent network. An RNN reads the sequence one element per time step; at step t it combines the new input with the previous state, so order is built into the computation and the model can handle any length. The **time step** is the unit of the unfolded computation and of back-propagation through time. **Parameter sharing across time** means the same W_{xh}, W_{hh}, W_{hy} are used at all steps: the parameter count (305) does not grow with L, a pattern learned at one position is recognised at another, and sequences of different lengths can be processed. **Application:** forecasting a sensor reading or classifying a vibration signal, speech or streaming text.

Student B — recursive network. A recursive network builds a representation bottom-up along a tree: leaves receive embeddings and each parent is computed from its children, $h_{parent} = \tanh(W[h_{left}; h_{right}] + b)$, with the same weights at every node, up to a root vector used for prediction. For T and (F or T) the node “F or T” is computed first and its vector is combined with “T” at the root, as drawn below. **Comparison:** sequential processing needs n steps and the path from the first to the last element has length n, whereas hierarchical composition needs only the tree depth (about log n for balanced trees) and mirrors the structure of the data, but it requires a tree (parse). **Application:** sentiment or meaning of a sentence over its parse tree, evaluating arithmetic or logical expressions, scene-parsing of nested objects.



Aspect	Recurrent network	Recursive network
Structure assumed	Linear order in time	Tree (hierarchy)
Computation	n sequential steps, one shared cell	One shared composition per tree node
Path between distant items	Up to n steps	Up to tree depth
Needs extra input	No	A tree or parser

Joint evaluation. A sequential RNN is more appropriate for a sentence when no reliable parse tree is available or the text is noisy and informal (tweets, speech transcripts), when words must be processed online as they arrive, or when meaning is dominated by word order and local context (for example short-sentence sentiment). The data then are a temporal stream whose structure is simply order, and building trees would add a parser whose errors propagate. When a correct syntactic tree exists and meaning depends on nested scope (negation applied to a clause), the recursive representation is the better match (Q5 gives measured evidence).

Q2 — Evaluating RNN performance using multiple metrics (7 marks)

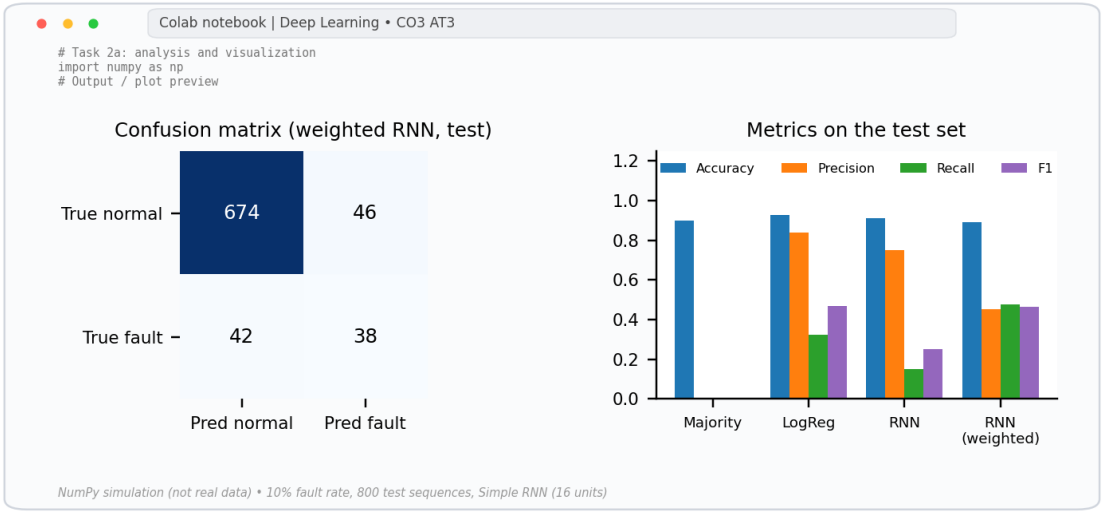
Shared group work

A Simple RNN (16 units, 321 parameters) classifies 30-step, 2-sensor sequences; a fault is a 5-step burst raising sensor 1 by 1.5 standard deviations, and faults make up only 80 of 800 test sequences (10%). Two baselines were set up: a **majority-class** model and a **logistic regression** on summary features (max, mean and standard deviation per sensor). The RNN was trained twice, with ordinary cross-entropy and with a positive-class weight of 6, using a separate validation set for early stopping and a single evaluation on the untouched test set.

Student A — metric investigation

Definitions (TP, FP, FN, TN from the confusion matrix): accuracy = (TP + TN)/N, the share of correct decisions; precision = TP/(TP + FP), how many raised alarms are real; recall = TP/(TP + FN), how many real faults are found; F1 = 2PR/(P + R), their harmonic mean, which is low if either is low. PR-AUC summarizes precision and recall over all thresholds. **Results.** The majority model scores 0.900 accuracy with zero recall. The unweighted RNN reaches 0.910 accuracy but only 0.150 recall (12 of 80 faults); weighting the positive class raises recall to 0.475 at the cost of precision (0.452), giving F1 = 0.463. The logistic baseline is as good (F1 0.468, PR-AUC 0.621), so the RNN is not yet justified.

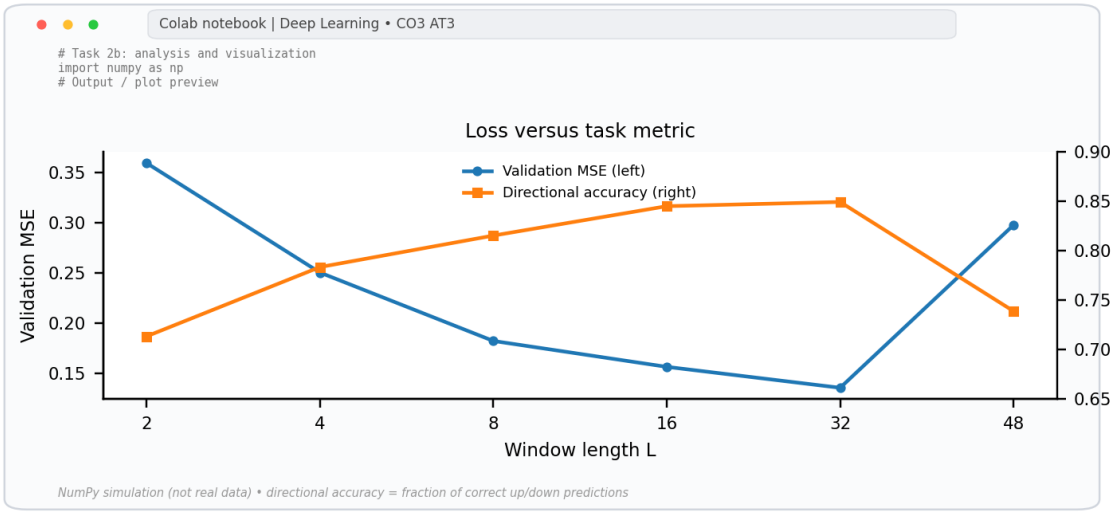
Model	Accuracy	Precision	Recall	F1	PR-AUC
Majority class	0.900	0.000	0.000	0.000	0.100
Logistic regression	0.926	0.839	0.325	0.468	0.621
RNN, unweighted	0.910	0.750	0.150	0.250	0.395
RNN, weighted (6×)	0.890	0.452	0.475	0.463	0.397



Training versus validation. For the unweighted RNN the loss is 0.343 (training) and 0.332 (validation) with F1 0.185 and 0.138; both are poor and close together, which is **underfitting of the minority class** (high bias), not overfitting. The weighted model shows F1 0.492 (train) against 0.424 (validation) and losses 0.506/0.538: a small gap and no sign of memorization, so the model is still limited by capacity, signal and training time rather than by overfitting. **Why one metric is not enough:** accuracy hides the minority class, precision and recall trade off against each other, and F1 hides which error type occurs and its cost.

Student B — sequence-level performance analysis

For the forecasting task the application objective is to anticipate whether the next reading rises or falls (for example to schedule maintenance), so alongside MSE the chosen metric is **directional accuracy**, the fraction of correct up/down predictions. The table of Q1 shows how both change with sequence length. **Loss falls but the task metric does not:** from L = 16 to L = 32 validation MSE improves by 13% (0.157 → 0.136) but directional accuracy moves only from 0.845 to 0.849, a gain of 0.4 percentage points, so the extra length and cost are not worthwhile. MSE rewards getting the magnitude right and is dominated by large deviations; directional accuracy measures the decision the application takes, so the metric should follow the objective.




```
from sklearn.metrics import confusion_matrix, classification_report, average_precision_score
p = model.predict(X_te).ravel(); pred = (p >= 0.5).astype(int)
print(confusion_matrix(y_te, pred)); print(classification_report(y_te, pred))
print("PR-AUC:", average_precision_score(y_te, p))
```

Joint evaluation. A model with 95% accuracy but poor minority-class performance is **not successful**: when the minority class is the event of interest (faults, disease), a trivial classifier can already reach 90–95% (here 90.0% with zero recall). Before accepting it we would require the confusion matrix with per-class precision, recall and F1; PR-AUC and a threshold analysis, because the 0.5 threshold is arbitrary; comparison against majority and simple baselines (here logistic regression); evidence on the cost of a missed event versus a false alarm; results on a time- or source-separated test set with confidence intervals; calibration of the probabilities; error analysis of the missed cases; and stability across seeds.

Q3 — Selecting hyperparameters for an RNN (7 marks)

Shared group work

A GRU-based classifier for the same fault sequences (30 steps, 2 sensors, faults 10%, positive-class weight 4) is the model under study. The initial configuration is learning rate 0.01, 16 hidden units, one recurrent layer, batch size 32, no dropout and 25 epochs. Every trial uses the same data split, seed and epoch budget so that only the named hyperparameter changes; the selection metric is **validation loss** (validation F1 as a tie-break) and the test set is used once for the chosen configuration.

Student A — hyperparameter design

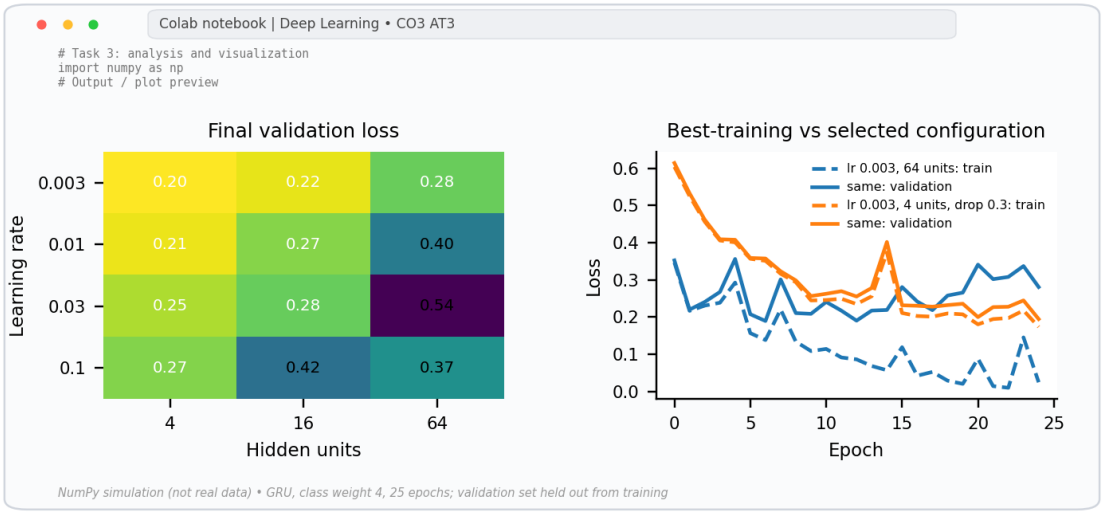
Hyperparameter	Candidates	Why it matters	Risk
Learning rate	0.003, 0.01, 0.03, 0.1	Sets the step size of every update	High: unstable; low: slow, underfit
Hidden units	4, 16, 64	Capacity of the state vector	Large: overfit; small: underfit
Recurrent layers	1, 2	Depth of temporal abstraction	More layers: harder to train, more overfit
Sequence length	Fixed 30 here (Q1: 2–48)	Context seen by the model	Too short: missing cues; too long: costly, unstable
Batch size	16, 32, 64	Gradient noise and speed	Small: noisy; large: poor generalization
Dropout	0, 0.3	Regularization on the final state	High: underfit

Controlled design. Stage 1 is a full grid over learning rate × hidden units (12 runs), because these two interact (a larger network needs a smaller step). Stage 2 takes the best pair and changes *one* further factor at a time (two layers, dropout 0.3, batch 16, batch 64). Likely **underfitting**: 4 hidden units with a very small learning rate, one short window, or heavy dropout. Likely **overfitting**: 64 hidden units with a moderate learning rate and no dropout, which can memorize the 1,200 training sequences.

Student B — hyperparameter evaluation

The 12 grid runs and four follow-up runs were executed and logged (training loss / validation loss, final epoch). **Learning rate:** 0.1 failed to learn (validation loss 0.267–0.417; F1 0.0–0.5), 0.03 with 64 units diverged (validation loss 0.542), while 0.003 and 0.01 trained smoothly. **Hidden size:** at learning rate 0.003 the training loss falls from 0.180 (4 units) to 0.024 (64 units) but the validation loss *rises* from 0.204 to 0.281: the large network overfits. **Convergence:** the 64-unit and 16-unit networks reached their best validation loss early (epochs 3–11) and then overfit, whereas the 4-unit networks improved slowly and were still improving at epochs 16–25. The best training performance (lr 0.003, 64 units, training loss 0.024) is **not** the best validation performance (0.281); the selected configuration is lr 0.003, 4 units, dropout 0.3 with training loss 0.173 and validation loss 0.194 (F1 0.667). On the untouched test set it gives accuracy 0.939, precision 0.663, recall 0.802 and F1 0.726.

Learning rate	4 units (train / val)	16 units (train / val)	64 units (train / val)
0.003	0.180 / 0.204	0.114 / 0.217	0.024 / 0.281
0.01	0.177 / 0.214	0.052 / 0.266	0.056 / 0.401
0.03	0.186 / 0.246	0.135 / 0.282	0.527 / 0.542
0.1	0.278 / 0.267	0.412 / 0.417	0.370 / 0.372
Follow-ups from best pair	2 layers: 0.231	dropout 0.3: 0.194	batch 16: 0.224
		batch 64: 0.210	




```
for lr in (0.003, 0.01, 0.03, 0.1):
  for units in (4, 16, 64):
    m = keras.Sequential([layers.Input(shape=(30, 2)), layers.GRU(units), layers.Dense(1, activation="sigmoid")])
    m.compile(keras.optimizers.Adam(lr), "binary_crossentropy"); h = m.fit(X_tr, y_tr, validation_data=(X_va, y_va),
epochs=25, class_weight={0: 1, 1: 4})
```

Joint design question. If the training performance is extremely good but performance on unseen sequences is poor, the first hyperparameters to examine are those that control capacity and regularization: the **number of hidden units** and layers (here 64 units gave a train–validation loss gap of 0.256 against 0.024 for 4 units), **dropout** and weight decay (dropout 0.3 improved validation loss from 0.204 to 0.194), the **number of epochs** with early stopping, and the **learning rate** and batch size that determine how closely the training set is fitted. Sequence length and the amount of data come next. Each is chosen by the gap between training and validation curves, not by the training loss.

Q4 — Debugging an RNN: diagnosing training failure (7 marks)

Shared group work

An intentionally problematic Simple RNN (32 units) is trained on 50-step fault sequences with a long sustained drift as the fault signature: its recurrent weights are drawn with gain 2.5 (spectral radius far above 1) and it is optimized with plain SGD at learning rate 1.0, class weight 4 and no clipping. The training behaviour was observed, causes were listed, a debugging procedure was followed and three modifications were verified against the same data and seed.

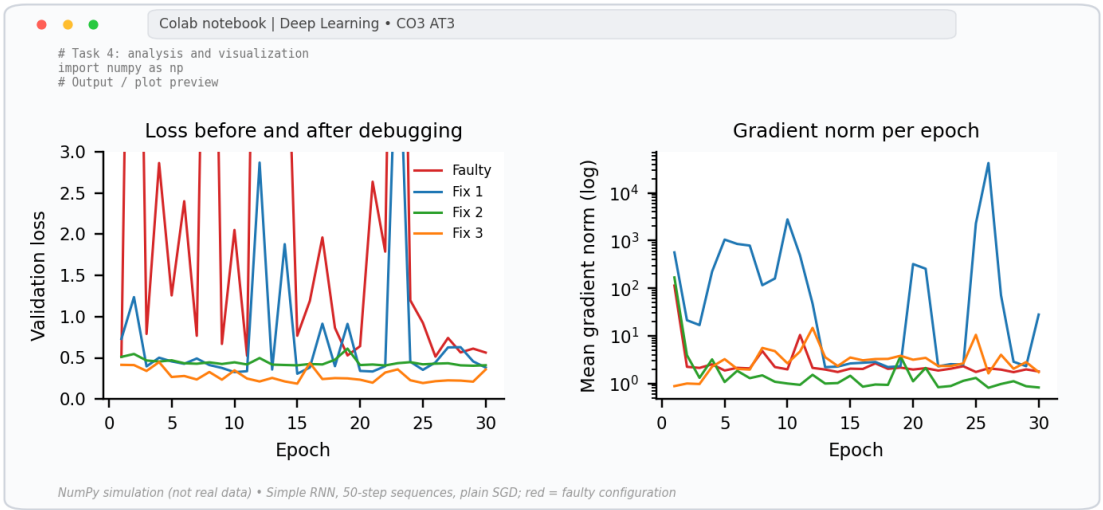
Student B — diagnosis and correctness

Symptoms seen: *exploding gradients* (mean gradient norm of the first epoch 113 with a maximum of 3,562), *unstable loss* (training loss jumps to 10.7 in the first ten epochs against about 0.4 in the later epochs), *no convergence* (validation F1 0.042). Vanishing gradients and severe overfitting would appear as near-zero gradient norms and a growing train–validation gap respectively, and were not the problem here. **Possible causes:** recurrent matrix with spectral radius above 1, learning rate too large for plain SGD, no clipping, unscaled inputs, label or mask bugs. **Gradient flow:** back-propagation through time multiplies the gradient at every step by $W_{hh}^{T}\,\mathrm{diag}(1 - h_t^2)$; if the largest singular value stays above 1 the product grows exponentially with the number of steps (explosion), if below 1 it shrinks (vanishing). **Checks:** numerical gradient check (passed: maximum difference about 10^{-10}), overfit a tiny batch, loss at initialization (should be near $\ln 2 = 0.693$ for balanced classes), per-step and per-layer gradient norms, input statistics and NaN scans. **Clipping** rescales the whole gradient to a maximum norm c, $g \leftarrow g \cdot c/|g|$, so one huge gradient cannot throw the weights far away, while preserving the direction. **Implementation error versus design problem:** an implementation error shows as a failed gradient check, inability to overfit a tiny batch or a wrong initial loss; here these checks pass, so the fault is a model-design/initialization problem. **Decision:** keep the architecture and correct the recurrent initialization together with clipping and a smaller step, rather than replacing the model.

Student A — experimental and engineering analysis

Four runs were compared over 30 epochs (training behaviour, validation behaviour, stability). The faulty run was reproduced first and its loss and gradient curves recorded; then three interventions were applied.

Configuration	Max train loss (ep. 1–10)	Max gradient norm	Final val loss	Final val F1
Faulty: gain 2.5, SGD η=1.0	10.71	3,562	0.564	0.042
Fix 1: clip norm 1.0	1.26	728,349	0.381	0.077
Fix 2: lower η = 0.05	0.54	3,562	0.410	0.048
Fix 3: orthogonal init + clip + η=0.1	0.45	391	0.358	0.500



Findings. *Clipping* removed the loss spike (1.26 instead of 10.7) but the model still did not learn the fault (F1 0.077), and the pre-clip norm reached 728,349, showing that the underlying instability remained. *A lower learning rate* gave a smooth loss but very slow learning (F1 0.048). *Orthogonal recurrent initialization with clipping and η = 0.1* kept the initial gradient norm near 1 (maximum over training 391, against 3,562 and 728,349), gave the

lowest validation loss (0.358) and by far the best F1 (0.500), so it was the **most effective intervention**. **Trade-offs:** clipping and orthogonal initialization cost almost nothing computationally; a smaller learning rate costs more epochs; switching to GRU or LSTM would triple or quadruple the recurrent parameters and increase training time about threefold in our runs (Q6, Q7).

```
m = keras.Sequential([layers.Input(shape=(50, 2)),
  layers.SimpleRNN(32, recurrent_initializer="orthogonal"), layers.Dense(1, activation="sigmoid")])
opt = keras.optimizers.SGD(0.1, clipnorm=1.0)                # clipping + smaller step
m.compile(opt, "binary_crossentropy"); m.fit(X_tr, y_tr, validation_data=(X_va, y_va), epochs=30, class_weight={0: 1, 1: 4})
```

Joint debugging decision (loss suddenly NaN). In order, before touching the architecture: (1) stop and restore the last finite checkpoint, keeping the batch that produced the NaN; (2) check the inputs of that batch for NaN/inf, extreme values and zero-length or fully masked sequences; (3) log the gradient norm of the preceding steps: a rapid increase indicates explosion, so add gradient clipping, lower the learning rate by about ten times and check the recurrent initialization; (4) inspect the loss for log(0), division by zero or overflow in exp (use the numerically stable logit form of cross-entropy and clip probabilities); (5) test in float64 and with a tiny batch to separate numerical precision from logic; (6) re-run the gradient check; (7) only if the NaN persists with a small learning rate, clipping and clean data, consider a model change. In our experiments the faulty configuration produced a loss spike of about 10 rather than an actual NaN, so these steps are the prescribed procedure rather than an observed outcome.

Q5 — RNN vs recursive neural network: model selection and evaluation (6 marks)

Shared group work

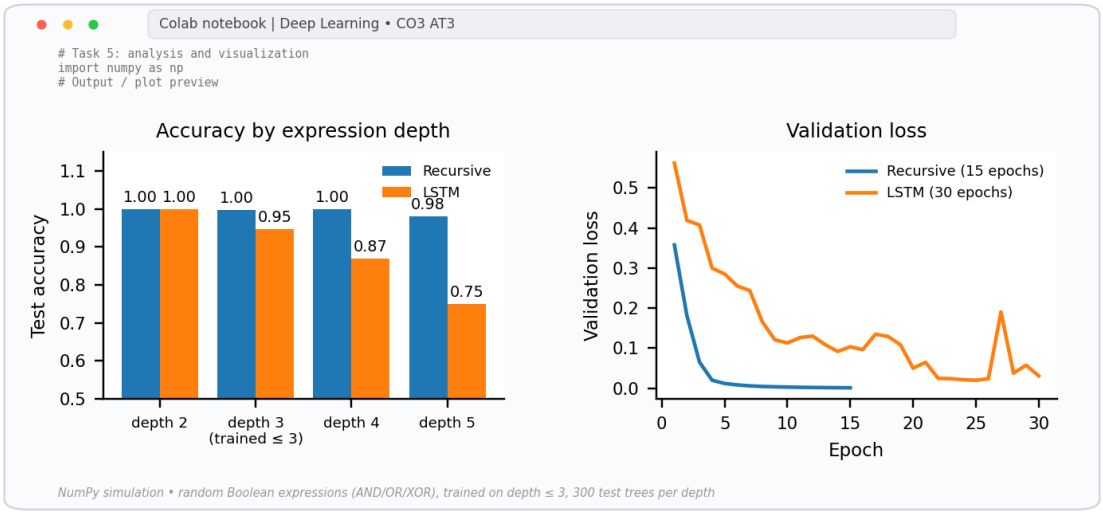
Structured problem: evaluating nested Boolean expressions such as (T and (F xor T)) with AND, OR and XOR, a task with exact hierarchical structure (label = value of the expression, about 50% true). Both models are trained on the same 1,500 trees of depth 1–3, validated on the same 300 trees and tested on 300 new trees of each depth 2 to 5, so depths 4 and 5 test whether the learned composition generalizes to deeper structure. Same loss (cross-entropy) and optimizer (Adam, $\eta = 0.01$). The RNN is an LSTM (24 units) over the token sequence; the recursive network is implemented directly (gradient-checked, maximum difference 1e-10).

Student A — architecture and representation design

RNN input: each expression is flattened to a one-hot token sequence over 7 symbols (T, F, and, or, xor, "(", ")"), so a depth-3 expression has about 23 tokens and a depth-5 one about 68; the final state is classified. **Recursive model:** leaves get a learned 16-dimensional embedding (T, F); each internal node computes $h = \tanh(W_l^{(op)} h_l + W_r^{(op)} h_r + b^{(op)})$ with operator-specific matrices; the root vector is classified. **Assumptions:** the RNN assumes only linear order and must learn bracket matching and scope itself; the recursive model assumes the correct tree is given. **Information preserved:** the RNN keeps token order and compresses everything into one state; the recursive model keeps the hierarchy and gives every sub-expression its own vector. **Computation:** the RNN needs one sequential step per token (68 for depth 5); the recursive model needs one composition per internal node (17 for depth 5) but needs a tree, and its nodes cannot be processed in a plain time-major batch.

Student B — comparative evidence

Measure	Recursive network	LSTM (RNN)
Trainable parameters	1,633	3,097
Training time (s) / epochs	1.1 / 15	6.1 / 30
Test accuracy, depth 2	1.000	1.000
Test accuracy, depth 3 (training depth)	0.997	0.947
Test accuracy, depth 4 (unseen)	1.000	0.870
Test accuracy, depth 5 (unseen)	0.980	0.750
Inference per tree (ms)	0.018	0.024



Training and validation behaviour: the recursive network's validation loss dropped faster and lower (final 0.001 against 0.031 for the LSTM) with half the parameters. **Strengths and weaknesses:** the recursive model generalizes almost perfectly to deeper trees (0.980 at depth 5) because the same composition is reused at every level, but it needs trees; the LSTM works directly on text-like sequences and batches well on hardware, yet its accuracy falls from 0.947 to 0.750 as expressions get longer than any seen in training. (Timings are single-CPU Python and not a fair hardware comparison; the epochs differ.) **Suitable model here:** the recursive network, because the data are an explicit hierarchy.

```
def compose(h_l, h_r, op):                    # shared weights per operator
  return np.tanh(h_l @ W_l[op] + h_r @ W_r[op] + b[op])
```

```
def encode(t): return E[t] if isinstance(t, int) else compose(encode(t[1]), encode(t[2]), t[0])
p = sigmoid(encode(tree) @ w_o + b_o)          # LSTM baseline: layers.LSTM(24) over token sequence
```

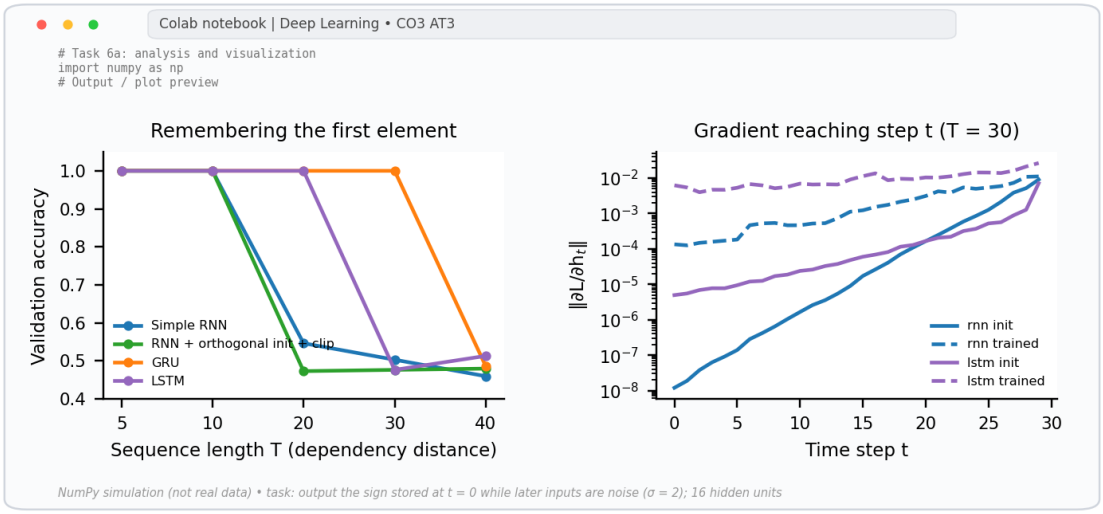
Cross-understanding. *Student B — why a recursive model helps for hierarchical input:* the meaning of a node is a function of its children only, so the model computes in the same order as the structure, the path between related parts is as short as the tree depth, and one composition function is shared across levels, which lets it handle deeper or differently shaped trees than were seen in training (accuracy 0.98 at depth 5 versus 0.75 for the LSTM). *Student A — evidence to justify the RNN over the recursive model:* equal or better held-out macro-F1 and validation loss under the same data and budget, robustness when trees come from an imperfect parser or do not exist, similar accuracy on out-of-structure and longer inputs, lower training and inference cost on available hardware, and results that are stable across seeds. In this experiment the evidence supports the recursive model; for streaming or unparsed text it would not.

Q6 — Advanced sequence-model mini-investigations (7 marks)

Student A — long-term dependency investigation

Task: the sign of the first element (t = 0) must be reported at the end; the remaining T − 1 inputs are noise (σ = 2), so the relevant information lies T − 1 steps back. **Baseline:** Simple RNN (16 units, Adam η = 0.01, 50 epochs, 600 training sequences). T was varied over 5, 10, 20, 30 and 40. **Evidence of difficulty:** the Simple RNN is perfect for T ≤ 10 but falls to chance level at T = 20 (0.547), 30 (0.503) and 40 (0.460). **Vanishing gradient:** the gradient of the loss with respect to the hidden state at step t shrinks as t moves away from the end; at T = 30 its norm is 1.3e-04 at t = 0 against 1.1e-02 at the last step for the trained Simple RNN, so the earliest input receives almost no learning signal. **Intervention:** gated cells. The GRU and LSTM keep an additive path ($c_t = f_t \cdot c_{t-1} + i_t \cdot g_t$ (element-wise products), with the forget bias initialized to 1) through which the gradient is multiplied by values near 1 instead of by W_{hh} and tanh′. **Before and after:** at T = 20 both gated models reach accuracy 1.0 (RNN 0.547); the GRU also reaches 1.0 at T = 30, whereas the LSTM did not train in that run (0.477), and both failed at T = 40 in the 50-epoch budget. Orthogonal initialization with gradient clipping did *not* help (0.473 at T = 20), since clipping only controls explosion, not vanishing. **Explanation:** gating improves the gradient path but does not remove optimization difficulty; the LSTM result at T = 30 and both failures at T = 40 show that more epochs, seeds or tuning would be needed before claiming reliability.

Dependency length T	Simple RNN	RNN + orthogonal + clip	GRU	LSTM
5	1.000	1.000	1.000	1.000
10	1.000	1.000	1.000	1.000
20	0.547	0.473	1.000	1.000
30	0.503	0.477	1.000	0.477
40	0.460	0.480	0.487	0.513



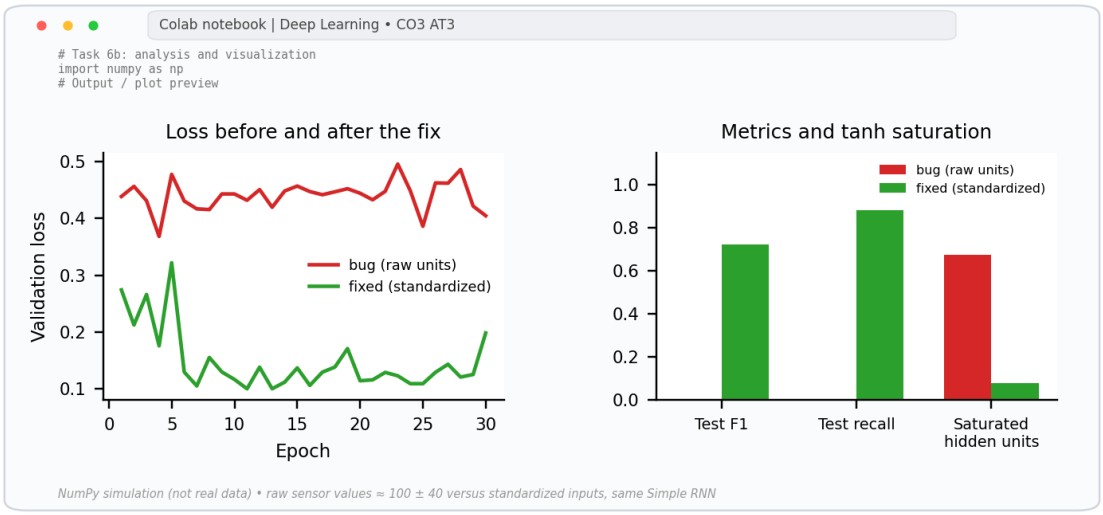

```
for T in (5, 10, 20, 30, 40):
  for cell in (layers.SimpleRNN, layers.GRU, layers.LSTM):
    m = keras.Sequential([layers.Input(shape=(T, 2)), cell(16), layers.Dense(1, activation="sigmoid")])
    m.compile(keras.optimizers.Adam(0.01), "binary_crossentropy", metrics=["accuracy"]); m.fit(X_tr, y_tr,
validation_data=(X_va, y_va), epochs=50)
```

Student B verification — why distance makes learning difficult. To learn from an early element, the gradient must pass backward through every intermediate step, being multiplied at each step by the recurrent weights and the activation derivative. A product of many factors smaller than 1 vanishes (and larger than 1 explodes), so the signal that links an early cause to the final output becomes too small to guide the weights; in addition, each new input overwrites part of the state, so the early information must survive many updates. Short-term dependencies are learned first and the long-term ones are missed.

Student B — debugging and model-refinement investigation

Controlled problem: the raw sensor values (about 100 ± 40 units) were fed to a GRU (16 units) without standardization, while the correct pipeline standardized them with training statistics; data, model, seed and

hyperparameters were otherwise identical. **Symptom:** the validation loss stayed at 0.405 (the loss of predicting the prior) and test F1 was 0.000 with recall 0, although accuracy looked acceptable (0.904). **Hypotheses:** (i) learning rate too small, (ii) architecture too weak, (iii) label or mask error, (iv) inputs outside the working range of the gates. **Debugging hypothesis:** large pre-activations drive sigmoid/tanh into saturation where the derivative is about zero; this predicts a high share of saturated hidden units, which was measured: 68% of hidden activations exceeded 0.95 in magnitude. **Correction:** standardize inputs. **Result:** validation loss 0.198, F1 0.724, recall 0.882, precision 0.615, accuracy 0.936, and saturation fell to 8%. **Source of the problem:** the **data preprocessing**, not the architecture, hyperparameters or training procedure, which is why only this change was needed.



Student A verification — why identify the source before changing several components. If the learning rate, cell type, size and preprocessing are all changed together, any improvement cannot be attributed to one cause, a real bug can be hidden by extra capacity, new bugs may be introduced and compute is wasted. Here switching to an LSTM or enlarging the model would not have removed the saturation, and the real cause would have stayed undiscovered; one controlled change at a time gives evidence that can be reproduced.

Q7 — Design and debug a sequence model from first principles (10 marks)

Design scenario and data

A company predicts a future event from variable-length sensor sequences (20–40 steps, 3 sensors) with noise (σ = 1), 12% missing values, 5% label noise and only 600 labelled training sequences (validation 250, test 500, event rate 25%). An **early marker** (a spike in the first 6 steps) must be combined with a **recent trend** (a drift in the last 10 steps) to produce an event, so both long-term and short-term dependencies matter. The initial RNN of the company is reproduced as the baseline. All data are synthetic.

Stage 1 — independent design proposals (retained)

Item	Student A (initial proposal)	Student B (initial proposal)
Architecture	Single-layer LSTM, 32 units	GRU, 24 units with a masking layer
Input and pre-processing	Standardize, replace missing values by 0, pad to 40 with zeros	Standardize with training statistics, add a missing-value indicator per sensor
Sequence handling	Fixed length 40, no mask	Variable length with masked padding, bucketing by length
Output	Sigmoid probability of the event	Sigmoid probability, threshold tuned on validation
Training strategy	Adam 0.01, batch 32, 40 epochs	Adam, clipping at 1.0, class weight, early stopping
Validation / hyperparameters	Hold-out set; grid of units and η	Stratified hold-out by sequence; small random search on F1/PR-AUC
Metrics	Accuracy, ROC-AUC	F1, recall, PR-AUC, confusion matrix
Failure sources / debugging	Vanishing gradients, noise, small data	Overfit a tiny batch, gradient check, norm logging, NaN scan

Both initial designs were actually run (5 seeds): Student A's LSTM gave test F1 0.412 with validation-loss instability 0.067; Student B's GRU gave F1 0.419 with instability 0.011; the company's baseline RNN gave F1 0.027.

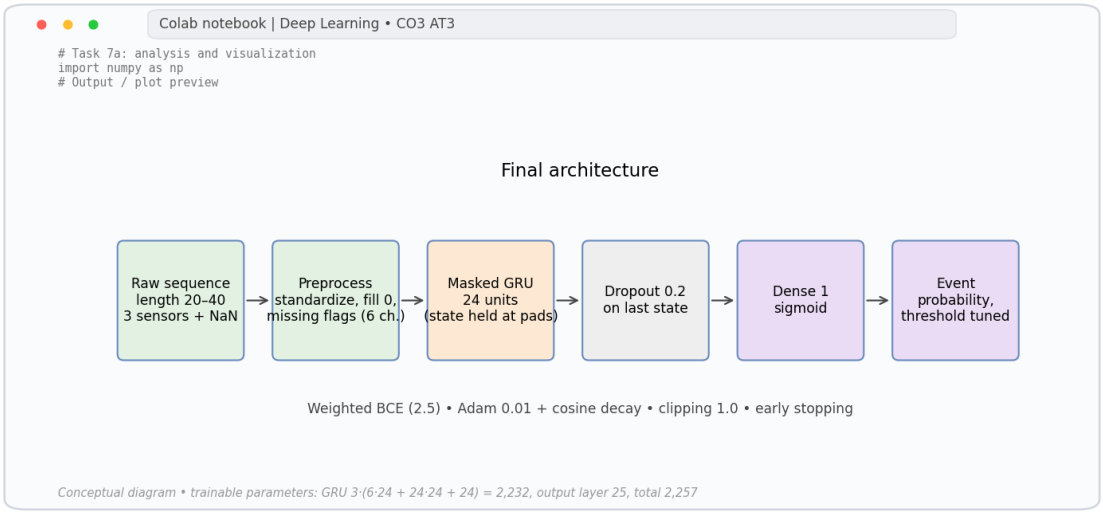
Stage 2 — design-comparison record

Aspect	Where the designs agree / differ	Decision for the final design
Cell	Both reject the plain RNN for gated cells; LSTM (A) versus GRU (B)	GRU: fewer parameters (2,257 vs 2,713) and same accuracy here
Padding / missing data	A ignores padding and fills with 0; B masks and flags	B's mask and indicators (Stage 4 shows the mask gives the largest gain)
Metrics	A: accuracy, ROC-AUC; B: F1, PR-AUC	F1, recall, PR-AUC and the confusion matrix; accuracy only as context
Training	B adds clipping, class weight, early stopping	Keep all of B's; add A's learning-rate schedule and dropout
Missing assumptions	Both treat missing values as random; neither addressed label noise	Document the assumption; use weighted loss and early stopping
Hyperparameters to investigate	Units, learning rate (both); dropout (A)	Units, learning rate, dropout, clipping norm, class weight

Stage 3 — joint final model design

Problem: binary prediction of a future event from a variable-length, noisy, partly missing multivariate sequence. **Input representation:** tensor (40, 6): three standardized sensor values (missing replaced by 0) and three

missing-value indicators. **Pre-processing:** statistics from the training set only; masks mark real steps. **Sequence length strategy:** variable lengths padded to 40 with a mask (longer sequences would keep the most recent 40 steps). **Architecture:** masked GRU layer with 24 units, dropout 0.2 on the last state, one dense sigmoid output (2,257 trainable parameters). **Recurrent processing:** $z_t = \sigma(W_z x_t + U_z h_{t-1})$, $r_t = \sigma(W_r x_t + U_r h_{t-1})$, $n_t = \tanh(W_n x_t + U_n(r_t \cdot h_{t-1}))$, $h_t = (1 - z_t) \cdot n_t + z_t \cdot h_{t-1}$; at padded steps $h_t = h_{t-1}$. **Hidden state:** the 24-dimensional state at the last real step. **Activations:** sigmoid gates, tanh candidate, sigmoid output. **Loss:** weighted binary cross-entropy (positive weight 2.5). **Metrics:** F1, recall, precision, PR-AUC, confusion matrix, plus validation-loss stability. **Hyperparameters:** Adam, η = 0.01 with cosine decay, batch 32, 40 epochs, clipping norm 1.0, dropout 0.2. **Training / validation:** stratified split by sequence, early stopping on validation loss with the best weights restored, threshold chosen on validation, test used once. **Debugging:** gradient check, tiny-batch overfit, padding-invariance test, gradient-norm and NaN monitoring. **Refinement:** change one component at a time and keep it only if validation and test results improve. A recursive model was not selected because the data have no tree structure.




```
for epoch in range(E):
  for batch in shuffled(train):                    # variable-length, padded, masked
    p = GRU_forward(batch.x, batch.mask); loss = weighted_BCE(p, batch.y, w1=2.5)
    g = BPTT(loss); g = clip_by_global_norm(g, 1.0); Adam_update(params, g, lr=cosine(epoch))
  if val_loss < best: save(params)                 # early stopping: restore the best weights
predict with threshold t* tuned on validation; evaluate once on the test set
```

```
model = keras.Sequential([layers.Input(shape=(40, 6)), layers.Masking(mask_value=-9.0),
  layers.GRU(24), layers.Dropout(0.2), layers.Dense(1, activation="sigmoid")])       # pad value -9, missing = 0
opt = keras.optimizers.Adam(keras.optimizers.schedules.CosineDecay(0.01, decay_steps=E*steps), clipnorm=1.0)
model.compile(opt, "binary_crossentropy"); model.fit(X_tr, y_tr, validation_data=(X_va, y_va), epochs=E, batch_size=32,
  class_weight={0: 1.0, 1: 2.5}, callbacks=[keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True)])
```

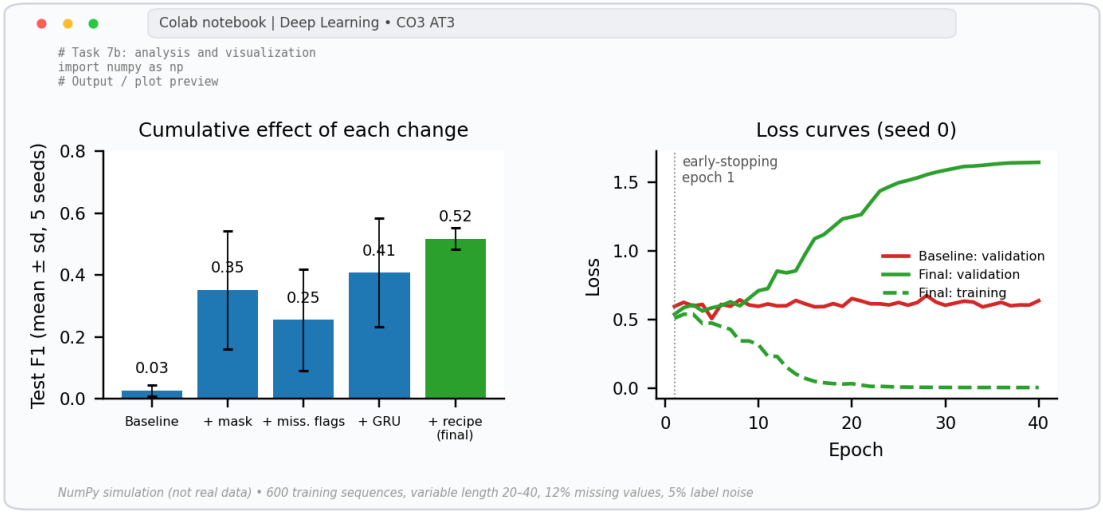
Stage 4 — validation by Student A and Student B

Student A (mathematical and performance validation). *Computational flow:* the input at each step passes through the update and reset gates, the candidate state and the interpolation above, so the state is a gated running summary; the final state goes through dropout and the sigmoid. *Components:* the mask makes the result independent of padding, the missing indicators let the model distinguish “zero” from “missing”, gating keeps a path for early information, clipping and the schedule control the update size. *Loss:* binary cross-entropy is the negative log-likelihood of Bernoulli labels and gives well-behaved gradients with a sigmoid; the positive weight compensates for the 25% event rate. *Metrics:* with 25% events accuracy is misleading, so F1, recall and PR-AUC are used. *Curves:* the baseline’s validation loss never gets below 0.49–0.60 and ends near 0.65, and its recall collapses to 0.015: an unstable, underfitting model that predicts almost no events. The final GRU shows the opposite problem: its training loss falls to about 0.003 by epoch 40 while its validation loss reaches a minimum of 0.52 in epoch 1–6 (across seeds) and then rises to 1.4–1.9, which is **strong overfitting** with only 600 sequences. **Early stopping with restored best weights is therefore essential** (all reported test results use the restored weights), and the low “instability” value of the final model measures only that its curve is smooth, not that it generalizes. *Against the baseline:* test F1 0.027 → 0.517 (± 0.035), recall 0.015 → 0.600, PR-AUC 0.281 → 0.446;

accuracy barely changes (0.713 → 0.699), which shows why it is not the deciding metric.

Student B (experimental and debugging validation). Correctness tests: (1) finite-difference gradient check for the RNN, GRU and LSTM cells with masks and two layers (maximum difference about 10^{-10}, *run*); (2) **padding invariance**: appending 10 more padded steps changes the final model's predictions by 0.0000 (none) but changes the baseline's by 0.0068 and flips 1.8% of its decisions, evidence of the "inconsistent predictions" reported by the company (*run*); (3) tiny-batch overfit, shape and NaN assertions (*to be executed in the Colab notebook*). The cumulative experiment below changes one component at a time (5 seeds; mean test F1 ± sd; instability = standard deviation of epoch-to-epoch validation-loss change over the last 15 epochs).

Configuration	F1 (mean ± sd)	PR-AUC	Recall	Accuracy	Instability	Time (s)
Baseline RNN (initial)	0.027 ± 0.018	0.281	0.015	0.713	0.041	1.5
+ padding mask	0.352 ± 0.191	0.443	0.360	0.721	0.042	1.9
+ missing indicators	0.255 ± 0.163	0.436	0.238	0.711	0.042	1.8
+ GRU cell	0.407 ± 0.175	0.495	0.391	0.730	0.031	4.9
+ training recipe (final)	0.517 ± 0.035	0.446	0.600	0.699	0.008	4.8
Student A initial (LSTM, no mask)	0.412 ± 0.058	0.429	0.374	0.716	0.067	4.9
Student B initial (GRU + mask + indicators)	0.419 ± 0.046	0.407	0.401	0.700	0.011	4.9



Largest improvements. The padding mask produced the largest single gain (F1 0.027 → 0.352); the GRU cell added 0.15; the training recipe (clipping, lower learning rate with decay, dropout, class weight and early stopping) added a further 0.11 and was the main source of stability: the spread across seeds fell from ± 0.175 to ± 0.035 and the instability from 0.031 to 0.008. The missing indicators alone did not help on average (0.352 → 0.255, within the seed noise), so that decision is supported by design logic rather than by this experiment; PR-AUC peaked at the GRU stage (0.495) and did not improve further.

Stage 5 — cross-defence

Student A — evidence that the final model is better than the original: on the same 500 test sequences and five seeds the final model reaches F1 0.517 against 0.027, recall 0.600 against 0.015 (the original finds almost no events: 2 of 136 in the confusion matrix, 103 of 136 for the final model with seed 0), PR-AUC 0.446 against 0.281, predictions that do not depend on padding, and validation-loss instability reduced by a factor of about five. **Student B — how the architecture and hyperparameters shape training and validation behaviour:** the high learning rate (0.05) and un-gated recurrence of the baseline gave erratic validation loss and a collapsed classifier; the gates and the additive state path let early information reach the output, clipping and the cosine schedule limit the update size, and the class weight prevents the classifier from settling on "no event". Overfitting is limited mainly by early stopping (best epoch 1–6); dropout 0.2 alone is not enough, since the training loss still reaches about 0.003. Part of the F1 gain also comes from the class weight, which moves predictions towards the event class at the 0.5 threshold;

the threshold-free PR-AUC improved from 0.281 to 0.446, but less than F1.

Stage 6 — joint evaluation

Why the architecture suits sequential data: the state carries information over time with weights shared across steps, handles variable length through the mask, and the gating retains both the early marker and the recent trend. **Why the metrics are appropriate:** they measure detection of the rare event and the cost balance between misses and false alarms, unlike accuracy. **Hyperparameters with the greatest effect:** learning rate and clipping (stability), hidden units and dropout (generalization), class weight (recall). **Evidence of overfitting/underfitting:** a rising validation loss with falling training loss indicates overfitting (the final GRU after epoch 1–6, training loss ≈ 0.003); high training and validation loss with a collapsed minority recall (the baseline) indicates underfitting or unstable learning. **Most effective debugging strategy:** verifying correctness first (gradient and padding-invariance checks) and then changing one component at a time, which isolated the padding bug and the unstable training. **Additional costs:** a GRU has three times the recurrent parameters of a Simple RNN and took about three times as long to train (4.8 s against 1.5 s per run), and recurrent steps cannot be parallelized over time. **When a recurrent model may not be appropriate:** very long sequences with dependencies spanning thousands of steps, data that are not ordered, or when parallel training speed is critical (attention or convolutional models are alternatives). **When a recursive model may be preferable:** when the data have an explicit tree (parse trees, expression trees, nested sub-systems), as in Q5, where it generalized to deeper structure. **How refinement improves generalization:** regularization, early stopping, correct preprocessing and data-aware masking reduce variance and remove artefacts the model could otherwise learn. **Next refinement to test:** stronger regularization (higher dropout, weight decay), fewer hidden units, noise/mask augmentation and more labelled data, each changed one at a time.

Conclusion — what a practitioner should examine before selecting, tuning and debugging a sequence model: the structure of the data (order, hierarchy, variable length, missing values, noise); the dependency range and the size and balance of the labelled data; how the split avoids leakage; the metric that reflects the real cost of errors and a simple baseline to beat; the input scale and preprocessing; the cell type and capacity appropriate to the dependency length; the hyperparameters with controlled, validation-based experiments; the gradient and loss behaviour (explosion, vanishing, NaN) and the correctness checks; the compute and latency budget; and finally whether the result is reproducible across seeds.

Contribution record and individual verification

The contribution record reflects the actual structure of the work. Primary contribution does not imply exclusive responsibility: both students remain accountable for the complete assignment and can explain every question, including the components for which they were not the primary contributor.

Question	Student A – primary contribution	Student B – primary contribution	Joint work
Q1	Sequence design and mathematical analysis	Experimental validation	RNN implementation and recurrent vs recursive comparison
Q2	Metric investigation	Sequence-level performance analysis	Model evaluation
Q3	Hyperparameter design	Hyperparameter evaluation	Hyperparameter selection
Q4	Experimental and engineering analysis	Debugging diagnosis	Debugging and refinement
Q5	Architecture and representation design	Comparative evidence	RNN vs recursive evaluation
Q6	Long-term dependency investigation	Debugging and model refinement	Cross-verification
Q7	Mathematical and performance validation	Experimental and debugging validation	Day 4 design challenge: complete model design, evaluation and defence

Reproducibility note. All numbers in this report come from the NumPy implementation run once with the stated seeds on synthetic data (the Q7 ablation uses five seeds). Before final submission both students should re-run the Keras excerpts in Google Colab on their chosen dataset and replace the panels with their own outputs; conclusions should be restated only where the new measurements agree.